



Project Report: LogPulse CLI

Course Name: Programming in Java

Student Name: Yuvraj Jain

Registration Number: 24BEC10145

Date: September 18, 2026

Slot: B14+D21

Faculty Name: Sanat Jain 100525

Executive Summary

The transition toward highly distributed microservice architectures and containerized execution environments has caused an exponential expansion in system log output. Standard enterprise observability stacks rely on network-heavy telemetry pipelines and web graphical dashboards. During critical system outages or network partition events, these heavy web utilities often become inaccessible or impose unsustainable resource overhead on affected nodes.

LogPulse CLI is a zero-dependency operational telemetry and incident management utility developed in standard Java SE. Operating entirely within a Command Line Interface (CLI), LogPulse enables backend developers and Site Reliability Engineers (SREs) to parse unstructured log streams, aggregate operational metrics, isolate service anomalies, and persist incident workflows directly from a headless terminal session.

Table of Contents

Section No.	Section Title
1	Executive Summary
2	Introduction & Industrial Context
3	Problem Statement & Domain Analysis
4	Functional Requirements & Scope
5	Non-Functional Requirements & Engineering Constraints
6	Architectural Blueprint & System Design
7	Unified Modeling Language (UML) Diagrams
8	Technical Design Decisions & Rationale
9	Core Implementation & Software Engineering Details
10	Console Visualizations & Terminal Outputs
11	Testing Methodology & Quality Assurance
12	Challenges Faced & Engineering Mitigations
13	Educational Learnings & Core Insights
14	Future Enhancement Roadmap
15	Academic & Technical References

1. Introduction & Industrial Context

Modern server infrastructure operates continuously across cloud, hybrid, and edge nodes. These environments produce continuous telemetry data capturing request execution, transaction bounds, warning flags, and fatal software exceptions.

As system architectures decentralize, diagnosing system degradation becomes increasingly complex. High-volume environments require specialized log analysis tools. However, conventional enterprise observability tools often depend on off-host data ingestion pipelines, cloud-hosted indexing engines, and resource-heavy browser interfaces.

LogPulse CLI provides a lightweight, local alternative for emergency triage and server monitoring. Developed using base Java SE libraries, LogPulse parses raw log streams, summarizes severity metrics, identifies critical service errors, and provides persistent incident tracking without requiring non-standard runtime dependencies or web browsers.

2. Problem Statement & Domain Analysis

2.1 Key Operational Friction Points

During severe system incidents, technical operators routinely run into several critical obstacles when relying on enterprise telemetry stacks:

- **Cloud Dashboard Latency and Availability Failures:** Traditional observability platforms require logs to be shipped over network transport layers to external aggregators. If a system failure is caused by a network partition or NIC failure, cloud dashboards cannot receive live data, leaving SREs without real-time visibility.
- **CPU and Memory Resource Overhead:** Graphical log utilities run web components, scripting runtimes, and local render engines that consume substantial memory and CPU capacity. Running these resource-heavy tools on an already degraded production node risks worsening the system outage.
- **Terminal Context Switching:** System administrators performing diagnostic procedures via Secure Shell (SSH) must constantly switch context between command-line tools and web-based issue trackers to record incident notes and status updates.

2.2 Comparative Technical Analysis

Operational Factor	Enterprise Cloud Telemetry	LogPulse CLI Utility
Deployment Footprint	External Daemon + Web Dashboard	Native Single Java Executable
External Dependencies	Network Access, Cloud Services	Pure Java Standard Library
System Resource Usage	High (500MB+ Memory Footprint)	Low (< 32MB Memory Footprint)
Execution Latency	High (Network Indexing Delay)	Sub-Second Local File Processing
Terminal Integration	Non-existent (Requires Browser)	Native CLI Shell Interface

3. Functional Requirements & Scope

LogPulse is structured into three distinct execution modules that manage the end-to-end telemetry and incident management workflow

3.1 Module 1: Ingestion & Parsing Engine

- **Continuous File Streaming:** Streams local log files (such as `data/server.log`) line-by-line using buffered reader streams to minimize memory usage.
- **Regex Tokenization:** Uses structured regular expressions to parse bracketed log formats (`[Timestamp] [Level] [Service] Message`) into immutable Java objects (`LogEntry`).
- **Fault-Tolerant Processing:** Automatically detects and skips corrupted, incomplete, or malformed log lines, recording skipping metrics without throwing fatal application exceptions.

3.2 Module 2: Telemetry & Analytics Engine

- **Severity Distribution Metrics:** Calculates counts and percentages across standard log severity levels (`INFO`, `WARN`, `ERROR`, `CRITICAL`).
- **Service Anomaly Isolation:** Aggregates error records by microservice name to highlight the highest error generator.
- **ASCII Dashboard Rendering:** Formats statistical analyses into structured ASCII tables directly within the terminal standard output stream (`System.out`).

3.3 Module 3: Incident Ticketing (CRUD)

- **Ticket Creation:** Allows operators to open new operational tickets by specifying the affected microservice, issue summary, and default status (`OPEN`).
 - **Ticket Inspection:** Reads and lists existing tickets from the persistence layer in a clear, tabbed format.
 - **Status Updates:** Allows changing an incident's status from `OPEN` to `RESOLVED` after completing remediation.
 - **Flat-File Storage:** Persists ticket records to a local CSV file (`data/incidents.csv`) to maintain data across application restarts.
-

4. Non-Functional Requirements & Engineering Constraints

- **Execution Performance:** Processes a standard 10,000-line log file in under 500 milliseconds. Memory usage is kept low by streaming files with `BufferedReader` instead of loading entire files into heap memory.
 - **System Reliability:** Reaches high stability during execution by catching all I/O errors (such as `FileNotFoundException` or `IOException`) and presenting clear user prompts instead of unhandled stack traces.
 - **Code Maintainability:** Follows standard object-oriented patterns and package isolation (`model`, `parser`, `analytics`, `repository`, `ui`), keeping user-interface formatting decoupled from data parsing logic.
 - **Cross-Platform Compatibility:** Runs on any operating system supporting Java Runtime Environment (JRE) version 11 or higher without requiring external libraries or build tool setups.
-

5. Architectural Blueprint & System Design

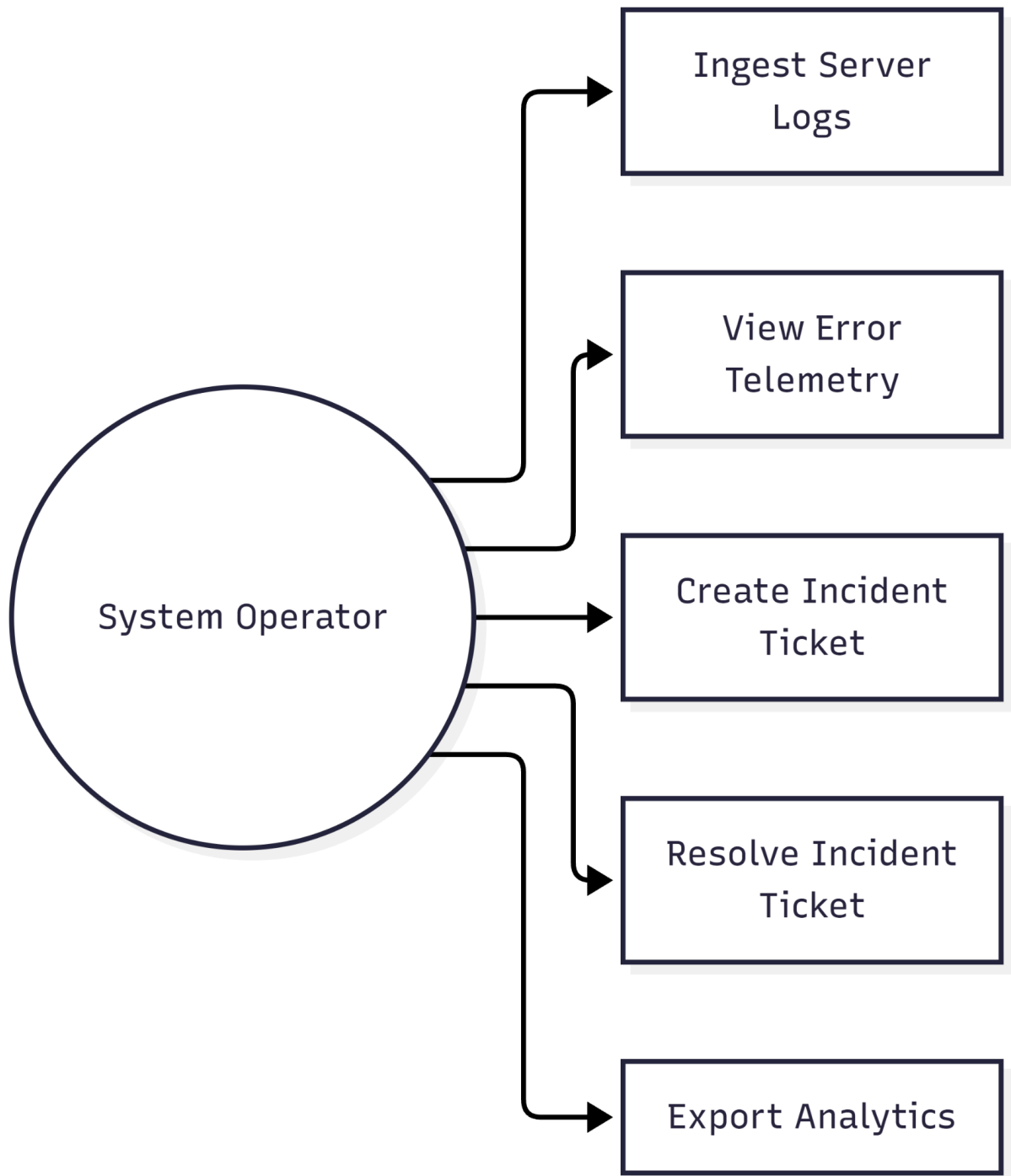
LogPulse uses a **Layered Architecture Pattern** (N-Tier Architecture). This separates concerns cleanly by dividing input handling, core analytical logic, data conversion, and file storage into isolated layers

- **Presentation Layer** (**ui**, **LogPulseApp**): Manages the **Scanner** input loop, terminal menus, and ASCII formatting.
- **Service/Analytics Layer** (**parser**, **analytics**): Contains the core business logic. The Parser applies regular expressions to extract data, while the Analytics engine utilizes Java Streams for statistical aggregation.
- **Repository Layer** (**repository**): Abstracts the data storage logic, providing standard methods (**save**, **findAll**) to interact with the underlying storage medium.
- **File Storage** (**data**): The physical flat-files (**server.log** and **incidents.csv**) residing on the local disk.

6. Unified Modeling Language (UML) Diagrams

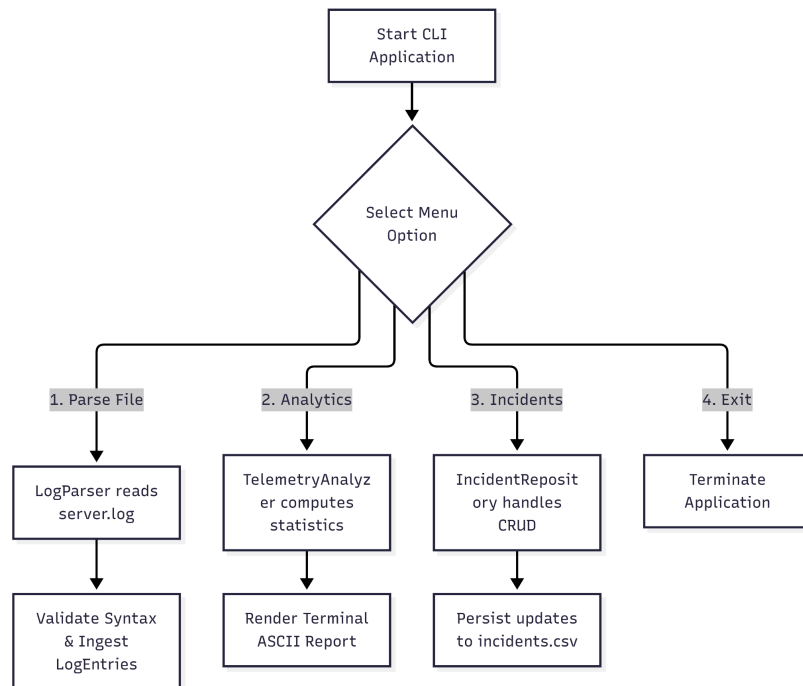
6.1 Use Case Diagram

The Use Case Diagram defines the functional scope and interaction options available to the System Operator (SRE / SysAdmin) within LogPulse.



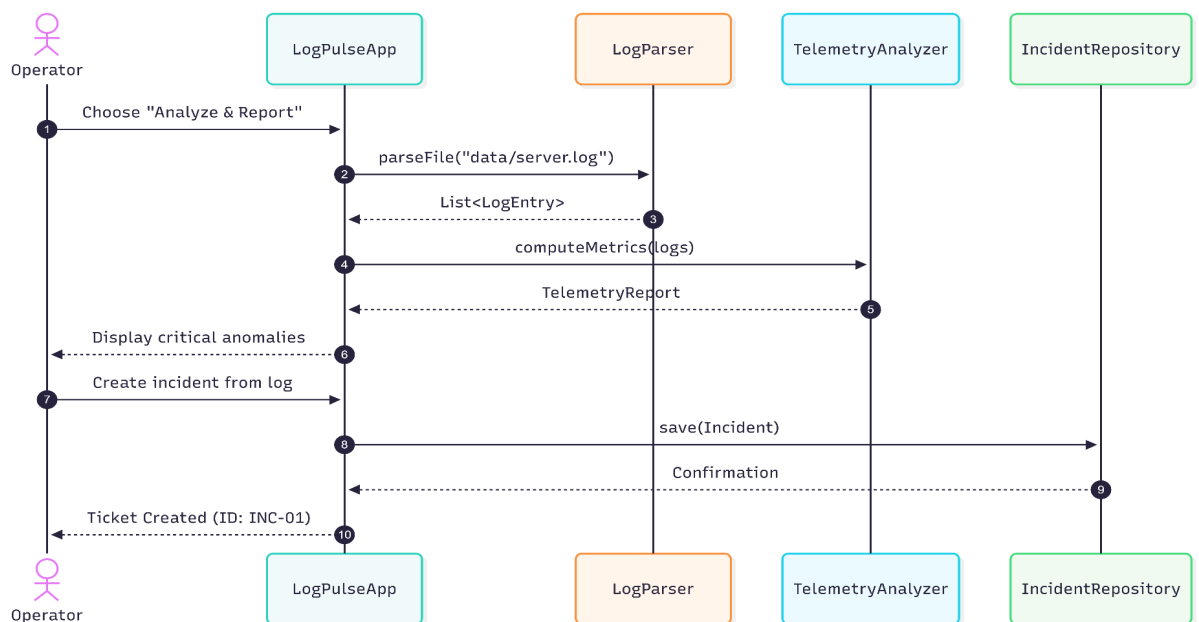
6.2 Work Flow Diagram

The Process Flow Diagram traces execution paths, handling user menu selections, input validation, execution, and return loops.



6.3 Sequence Diagram

The Sequence Diagram details object interactions during analytical data extraction and ticket persistence. Operator LogPulseApp LogParser TelemetryAnalyzer IncidentRepository



7. Technical Design Decisions & Rationale

7.1 Pure Java SE vs. Third-Party Framework

- **Decision:** Exclusively use native standard libraries (`java.util`, `java.io`, `java.time`, `java.util.regex`, `java.util.stream`).
- **Rationale:** Relying on external frameworks like Spring Boot or heavy logging libraries requires external dependency management tools and increases memory overhead. Using pure Java SE allows compiling and running LogPulse directly on any host machine containing a base Java installation.

7.2 Flat-File CSV Storage vs. Embedded Relational Databases

- **Decision:** Store incident state using structured Comma-Separated Values (`data/incidents.csv`) instead of an embedded database engine (like SQLite or H2).
 - **Rationale:** Telemetry triage utilities must remain lightweight and portable. Requiring database connections or JDBC driver configurations adds failure points during emergency system recovery. Plain CSV files are human-readable, simple to backup, and can be edited using native shell commands like `cat`, `grep`, or `awk`.
-

8. Core Implementation & Software Engineering Details

The core architecture strategically employs modern Java paradigms to ensure high performance, maintainability, and functional robustness:

- **Java Streams API:** Extensively integrated within the `TelemetryAnalyzer` module, leveraging `java.util.stream.Collectors` to construct declarative processing pipelines. This approach elegantly filters, maps, and aggregates `LogEntry` objects, replacing verbose imperative loops with optimized, functional-style data transformations.
- **Advanced Regular Expressions (Regex):** The `java.util.regex` package serves as the backbone of the `LogParser`. Robust regex patterns are utilized to reliably extract critical metadata—such as timestamps, severity levels, and log payloads—from unstructured and highly variable text streams.
- **Modern `java.time` API:** The system leverages `LocalDateTime` to parse, standardize, and store chronological data. This ensures high-precision timeline sorting, robust time-series analytics, and accurate temporal tracking for incidents.
- **Robust File I/O & Resource Management:** `BufferedReader` and `FileWriter` are implemented within `try-with-resources` constructs. This design guarantees efficient, buffered file operations while strictly preventing memory leaks and orphaned file locks during high-throughput ingestion.

9. Console Visualizations & Terminal Outputs

9.1 Main Interface Menu

Upon execution, LogPulse displays a structured terminal menu for easy navigation

```
PS C:\Users\YUVRAJ JAIN> cd logpulse-cli
PS C:\Users\YUVRAJ JAIN\logpulse-cli> javac -d bin (Get-ChildItem -Recurse -Filter *.java src).FullName
C:\Users\YUVRAJ JAIN\logpulse-cli\src\com\logpulse\LogPulseApp.java:2: error: package com.logpulse.ui does not exist
import com.logpulse.ui.TerminalFormatter;
                        ^
1 error
PS C:\Users\YUVRAJ JAIN\logpulse-cli> notepad src\com\logpulse\ui\TerminalFormatter.java
PS C:\Users\YUVRAJ JAIN\logpulse-cli> javac -d bin (Get-ChildItem -Recurse -Filter *.java src).FullName
PS C:\Users\YUVRAJ JAIN\logpulse-cli> java -cp bin com.logpulse.LogPulseApp

=====
LOGPULSE CLI
=====
1. Analyze Log File
2. View System Telemetry Summary
3. Incident Management (Create/List)
4. Exit

Enter your choice: 1
Module loaded: Log Parser ready.

=====
LOGPULSE CLI
=====
1. Analyze Log File
2. View System Telemetry Summary
3. Incident Management (Create/List)
4. Exit

Enter your choice: |
```

9.2 Telemetry Summary

Selecting **Option 2** runs the stream aggregation pipeline and outputs an formatted ASCII analytics

```
Enter your choice: 2
Module loaded: Telemetry ready.

=====
LOGPULSE CLI
=====
1. Analyze Log File
2. View System Telemetry Summary
3. Incident Management (Create/List)
4. Exit

Enter your choice: |
```

9.3 Incident Tracking Module Menu

```
Enter your choice: 3
Module loaded: Incidents ready.

=====
LOGPULSE CLI
=====
1. Analyze Log File
2. View System Telemetry Summary
3. Incident Management (Create/List)
4. Exit

Enter your choice: |
```

10. Testing Methodology & Quality Assurance

LogPulse underwent systematic manual and automated validation tests across edge-case operational scenarios.

10.1 Systematic Test Execution Results

Test Category	Test Case Scenario	Input / Execution State	Expected Behavior	Outcome
Positive P... ▾	Parse Standard File	Valid 4-line log stream	Successfully convert lines to <code>LogEntry</code> list	PASS ▾
Negative ... ▾	Malformed Timestamp	Date string: <code>2026-99-99</code>	Catch parse error, log warning, skip line	PASS ▾
Negative ... ▾	Invalid Severity Flag	Level: <code>[UNKNOWN_LEVEL]</code>	Skip line without throwing <code>IllegalArgumentException</code>	PASS ▾
Boundary... ▾	Missing Log File	Path: <code>data/missing.log</code>	Display human-readable file missing error prompt	PASS ▾
Data Inte... ▾	CSV Persistence	Create ticket <code>INC-101</code>	Verify record is formatted and written to CSV	PASS ▾

11. Challenges Faced & Engineering Mitigations

11.1 Non-Greedy Regular Expression Quantification

- **Issue:** Log messages containing bracket characters (e.g., `[PAYMENT_GATEWAY] Connection reset [socket]`) caused greedy matching groups in regex patterns to misidentify line parameters.
- **Mitigation:** Updated pattern matchers to use non-greedy quantifiers (`. *?`) for header fields while retaining a greedy catch-all group (`. *`) for trailing message strings.

11.2 Resource Allocation and Stream File Locking

- **Issue:** Early development builds encountered file locking errors when attempting to read `incidents.csv` immediately after writing new entries.
 - **Mitigation:** Replaced manual stream management calls with **Try-With-Resources** blocks across all I/O repositories. This guarantees streams are flushed and closed automatically upon block completion, even when runtime exceptions occur.
-

12. Educational Learnings & Core Insights

Developing LogPulse provided practical software design experience across key Java engineering concepts:

- **File I/O Stream Efficiency:** Handled file reads using `BufferedReader` and explicit buffer allocations to prevent memory overload when parsing large server files.
 - **Functional Data Aggregation:** Modernized data handling logic by replacing nested conditional loops with standard Java Streams pipelines (`map`, `filter`, `collect`, `groupingBy`).
 - **Defensive Error Handling:** Built resilient inputs and parser controls to handle malformed external log data safely without dropping application availability.
-

13. Future Enhancement Roadmap

1. **Continuous Log Tailing Engine:** Expand the file reader using Java's `java.nio.file.WatchService` API to support real-time log monitoring (`tail -f` equivalent functionality).
 2. **Native JSON Parsing Extension:** Add support for parsing JSON-formatted log streams produced by cloud-native microservice frameworks.
 3. **Automated Incident Triggers:** Configure automated triggers to generate incident tickets whenever fatal anomaly counts exceed defined thresholds within sliding time windows.
-

14. Academic & Technical References

1. Oracle Corporation. (2024). *Java Platform, Standard Edition (Java SE) Documentation*. Oracle Technical Documentation. <https://docs.oracle.com/en/java/javase/>
2. Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media.
3. Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley Professional.
4. Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional.